Building a Binary Classification Model in PyTorch

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch** 1

PyTorch library is for deep learning. Some applications of deep learning models are to solve regression or classification problems. In this post, you will discover how to use PyTorch to develop and evaluate neural network models for binary classification problems.

After completing this post, you will know:

- How to load training data and make it available to PyTorch
- How to design and train a neural network
- How to evaluate the performance of a neural network model using k-fold cross validation
- How to run a model in inference mode
- How to create receiver operating characteristics curve for a binary classification model

Kick-start your project with my book Deep Learning with PyTorch. It provides **self-study tutorials** with **working code**.

Let's get started.

Building a Binary Classification Model in PyTorch
Photo by David Tang. Some rights reserved.

Description of the Dataset

The dataset you will use in this tutorial is the Sonar dataset.

This is a dataset that describes sonar chirp returns bouncing off different services. The 60 input variables are the strength of the returns at different angles. It is a binary classification problem that requires a model to differentiate rocks from metal cylinders.

You can learn more about this dataset on the UCI Machine Learning repository. You can download the dataset for free and place it in your working directory with the filename `sonar.csv`.

It is a well-understood dataset. All the variables are continuous and generally in the range of 0 to 1. The output variable is a string "M" for mine and "R" for rock, which will need to be converted to integers 1 and 0.

A benefit of using this dataset is that it is a standard benchmark problem. This means that we have some idea of the expected skill of a good model. Using cross-validation, a neural network should be able to achieve a performance of 84% to 88% accuracy.

Load the Dataset

If you have downloaded the dataset in CSV format and saved it as `sonar.csv` in the local directory, you can load the dataset using pandas. There are 60 input variables (X) and one output variable (y). Because the file contains mixed data of strings and numbers, it is easier to read them using pandas rather than other tools such as NumPy.

Data can be read as follows:

```
import pandas as pd

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]
```

It is a binary classification dataset. You would prefer a numeric label over a string label. You can do such conversion with `LabelEncoder` in scikit-learn. The `LabelEncoder` is to map each label to an integer. In this case, there are only two labels and they will become 0 and 1.

Using it, you need to first call the `fit()` function to make it learn what labels are available. Then call `transform()` to do the actual conversion. Below is how you use `LabelEncoder` to convert `y` from strings into 0 and 1:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
```

You can see the labels using:

```
print(encoder.classes_)
```

which outputs:

```
['M' 'R']
```

and if you run `print(y)`, you would see the following

```
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

You see the labels are converted into 0 and 1. From the `encoder.classes_`, you know that 0 means "M" and 1 means "R". They are also called the negative and positive classes respectively in the context of binary classification.

Afterward, you should convert them into PyTorch tensors as this is the format a PyTorch model would like to work with.

```
import torch

X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
```

Creating a Model

Now you're ready for the neural network model.

As you have seen in some previous posts, the easiest neural network model is a 3-layer model that has only one hidden layer. A deep learning model is usually referring to those with more than one hidden layer. All neural network models have parameters called weights. The more parameters a model has, heuristically we believe that it is more powerful. Should you use a model with fewer layers but more parameters on each layer, or a model with more layers but less parameters each? Let's find out.

A model with more parameters on each layer is called a wider model. In this example, the input data has 60 features to predict one binary variable. You can assume to make a wide model with one hidden layer of 180 neurons (three times the input features). Such model can be built using PyTorch:

```
import torch.nn as nn

class Wide(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(60, 180)
        self.relu = nn.ReLU()
        self.output = nn.Linear(180, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden(x))
        x = self.sigmoid(self.output(x))
        return x
```

Because it is a binary classification problem, the output have to be a vector of length 1. Then you also want the output to be between 0 and 1 so you can consider that as probability or the model's confidence of prediction that the input corresponds to the "positive" class.

A model with more layer is called a deeper model. Considering that the previous model has one layer with 180 neurons, you can try one with three layers of 60 neurons each instead. Such model can be built using PyTorch:

```
class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x
```

You can confirm that these two models are having similar number of parameters, as follows:

```
# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()]))  # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()]))  # 11041
```

There will be all the model's parameters returned by `model1.parameters()` and each is a PyTorch tensors. Then you can reformat each tensor into a vector and count the length of the vector, using `x.reshape(-1).shape[0]`. So the above sum up the total number of parameters in each model.

Comparing Models with Cross-Validation

Should you use a wide model or a deep model? One way to tell is to use cross-validation to compare them.

It is a technique that, use a "training set" of data to train the model and then use a "test set" of data to see how accurate the model can predict. The result from test set is what you should focus on. But you do not want to test a model once because if you see an extremely good or bad result, it may be by chance. You want to run this process k times with different training and test sets, such that you are ensured that you are comparing the **model design**, not the result of a particular training.

The technique that you can use here is called k-fold cross validation. It is to split a larger dataset into k portions and take one portion as the test set while the $k-1$ portions are combined as the training set. There are k different such combinations. Therefore you can repeat the experiment for k times and take

the average result.

In scikit-learn, you have a function for stratified k-fold. Stratified means that when the data is split into k portions, the algorithm will look at the labels (i.e., the positive and negative classes in a binary classification problem) to ensure it is split in such a way that each portion contains equal number of either classes.

Running k-fold cross validation is trivial, such as the following:

```
# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores.append(acc)

# evaluate the model
acc = np.mean(cv_scores)
std = np.std(cv_scores)
print("Model accuracy: %.2f%% (+/- %.2f%%)" % (acc*100, std*100))
```

Simply speaking, you use `StratifiedKFold()` from scikit-learn to split the dataset. This function returns to you the indices. Hence you can create the splitted dataset using `X[train]` and `X[test]` and named them training set and validation set (so it is not confused with "test set" which will be used later, after we picked our model design). You assume to have a function that runs the training loop on a model and give you the accuracy on the validation set. You can than find the mean and standard deviation of this score as the performance metric of such model design. Note that you need to create a new model every time in the for-loop above because you should not re-train a trained model in the k-fold cross valiation.

The training loop can be defined as follows:

```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 250   # number of epochs to run
    batch_size = 10  # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf   # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_val)
        acc = (y_pred.round() == y_val).float().mean()
        acc = float(acc)
        if acc > best_acc:
            best_acc = acc
            best_weights = copy.deepcopy(model.state_dict())
    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    return best_acc
```

The training loop above contains the usual elements: The forward pass, the backward pass, and the gradient descent weight updates. But it is extended to have an evaluation step after each epoch: You run the model at evaluation mode and check how the model predicts the **validation set**. The accuracy on

the validation set is remembered along with the model weight. At the end of the training, the best weight is restored to the model and the best accuracy is returned. This returned value is the best you ever encountered during the many epochs of training and it is based on the validation set.

Note that you set `disable=True` in the `tqdm` above. You can set it to `False` to see the training set loss and accuracy as you progress in the training.

Remind that the goal is to pick the best design and train the model again, which in the training, you want to have an evaluation score so you know what to expect in production. Thus you should split the entire dataset you obtained into a training set and test set. Then you further split the training set in k-fold cross validation.

With these, here is how you can compare the two model designs: By running k-fold cross validation on each and compare the accuracy:

```
from sklearn.model_selection import StratifiedKFold, train_test_split

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))
```

You may see the output of above as follows:

```
Accuracy (wide): 0.72
Accuracy (wide): 0.66
Accuracy (wide): 0.83
Accuracy (wide): 0.76
Accuracy (wide): 0.83
Accuracy (deep): 0.90
Accuracy (deep): 0.72
Accuracy (deep): 0.93
Accuracy (deep): 0.69
Accuracy (deep): 0.76
Wide: 75.86% (+/- 6.54%)
Deep: 80.00% (+/- 9.61%)
```

So you found that the deeper model is better than the wider model, in the sense that the mean accuracy is higher and its standard deviation is lower.

Retrain the Final Model

Now you know which design to pick, you want to rebuild the model and retrain it. Usually in k-fold cross validation, you will use a smaller dataset to make the training faster. The final accuracy is not an issue because the gold of k-fold cross validation to to tell which design is better. In the final model, you want to provide more data and produce a better model, since this is what you will use in production.

As you already split the data into training and test set, these are what you will use. In Python code,

```
# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")
```

You can reuse the `model_train()` function as it is doing all the required training and validation. This is because the training procedure doesn't change for the final model or during k-fold cross validation.

This model is what you can use in production. Usually it is unlike training, prediction is one data sample at a time in production. The following is how we demonstate using the model for inference by running five samples from the test set:

```
model.eval()
```

```
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")
```

Its output should look like the following:

```
[0.0265 0.044  0.0137 0.0084 0.0305 0.0438 0.0341 0.078  0.0844 0.0779
 0.0327 0.206  0.1908 0.1065 0.1457 0.2232 0.207  0.1105 0.1078 0.1165
 0.2224 0.0689 0.206  0.2384 0.0904 0.2278 0.5872 0.8457 0.8467 0.7679
 0.8055 0.626  0.6545 0.8747 0.9885 0.9348 0.696  0.5733 0.5872 0.6663
 0.5651 0.5247 0.3684 0.1997 0.1512 0.0508 0.0931 0.0982 0.0524 0.0188
 0.01   0.0038 0.0187 0.0156 0.0068 0.0097 0.0073 0.0081 0.0086 0.0095] -> [0.9583146] (expected [1.])
...

[0.034  0.0625 0.0381 0.0257 0.0441 0.1027 0.1287 0.185  0.2647 0.4117
 0.5245 0.5341 0.5554 0.3915 0.295  0.3075 0.3021 0.2719 0.5443 0.7932
 0.8751 0.8667 0.7107 0.6911 0.7287 0.8792 1.     0.9816 0.8984 0.6048
 0.4934 0.5371 0.4586 0.2908 0.0774 0.2249 0.1602 0.3958 0.6117 0.5196
 0.2321 0.437  0.3797 0.4322 0.4892 0.1901 0.094  0.1364 0.0906 0.0144
 0.0329 0.0141 0.0019 0.0067 0.0099 0.0042 0.0057 0.0051 0.0033 0.0058] -> [0.01937182] (expected [0.])
```

You run the code under `torch.no_grad()` context because you sure there's no need to run the optimizer on the result. Hence you want to relieve the tensors involved from remembering how the values are computed.

The output of a binary classification neural network is between 0 and 1 (because of the sigmoid function at the end). From `encoder.classes_`, you can see that 0 means "M" and 1 means "R". For a value between 0 and 1, you can simply round it to the nearest integer and interpret the 0-1 result, i.e.,

```
y_pred = model(X_test[i:i+1])
y_pred = y_pred.round() # 0 or 1
```

or use any other threshold to quantize the value into 0 or 1, i.e.,

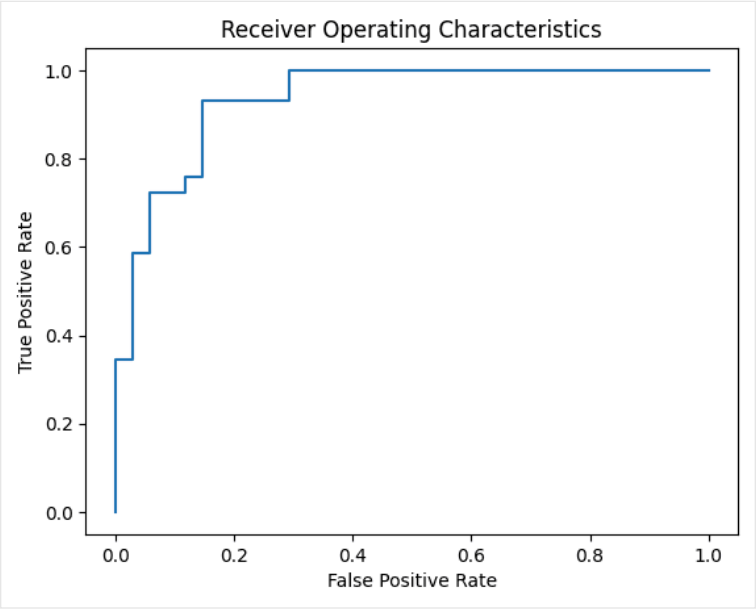
```
threshold = 0.68
y_pred = model(X_test[i:i+1])
y_pred = (y_pred > threshold).float() # 0.0 or 1.0
```

Indeed, round to the nearest integer is equivalent to using 0.5 as the threshold. A good model should be robust to the choice of threshold. It is when the model output exactly 0 or 1. Otherwise you would prefer a model that seldom report values in the middle but often return values close to 0 or close to 1. To see if your model is good, you can use **receiver operating characteristic curve** (ROC), which is to plot the true positive rate against the false positive rate of the model under various threshold. You can make use of scikit-learn and matplotlib to plot the ROC:

```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

with torch.no_grad():
    # Plot the ROC curve
    y_pred = model(X_test)
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
    plt.title("Receiver Operating Characteristics")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.show()
```

You may see the following. The curve is always start from the lower left corner and ends at upper right corner. The closer the curve to the upper left corner, the better your model is.



Complete Code

Putting everything together, the following is the complete code of the above:

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.metrics import roc_curve
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import LabelEncoder

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define two models
class Wide(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(60, 180)
        self.relu = nn.ReLU()
        self.output = nn.Linear(180, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden(x))
        x = self.sigmoid(self.output(x))
        return x

class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x

# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()]))  # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()]))  # 11041

# Helper function to train one model
def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss()  # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 300   # number of epochs to run
    batch_size = 10  # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf   # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
```

```
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_val)
        acc = (y_pred.round() == y_val).float().mean()
        acc = float(acc)
        if acc > best_acc:
            best_acc = acc
            best_weights = copy.deepcopy(model.state_dict())
    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    return best_acc

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))

# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

    # Plot the ROC curve
    y_pred = model(X_test)
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
    plt.title("Receiver Operating Characteristics")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.show()
```

Summary

In this post, you discovered the use of PyTorch to build a binary classification model.

You learned how you can work through a binary classification problem step-by-step with PyTorch, specifically:

- How to load and prepare data for use in PyTorch
- How to create neural network models and use k-fold cross validation to compare them
- How to train a binary classification model and obtain the receiver operating characteristics curve for it

More On This Topic

Binary Classification Tutorial with the Keras Deep…

How To Work Through a Binary Classification Project…

Add Binary Flags for Missing Values for Machine Learning

Building a Multiclass Classification Model in PyTorch

Building a Regression Model in PyTorch

Building a Logistic Regression Classifier in PyTorch

About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.
View all posts by Adrian Tam →

‹ Building a Multiclass Classification Model in PyTorch

Building a Regression Model in PyTorch ›

One Response to *Building a Binary Classification Model in PyTorch*

shadow February 5, 2024 at 5:26 pm #

REPLY

Hello, there is a problem I don't understand. In the two loops of fold.split, there is model=Wide() or Deep() at the beginning of each loop. Is this a reset of the model? If each loop continues to train the best model in the previous cycle, Whether model=Wide() or Deep() should be placed outside the loop to avoid reassignment of the model

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
Read more

Never miss a tutorial:

Picked for you:

PyTorch Tutorial: How to Develop Deep Learning Models with Python

LSTM for Time Series Prediction in PyTorch

Deep Learning with PyTorch (9-Day Mini-Course)


Calculating Derivatives in PyTorch


Develop Your First Neural Network with PyTorch, Step by Step